MiniHV: relatório do escopo 09

David dos Santos Oliveira (matrícula 2611820)

1. O escopo

O escopo 09 pede o escalonador CREDIT com quantum 5, um host com 4 pCPUs e no máximo 6 VMs admitidas ao mesmo tempo, memória contígua (modelo CONT) com BEST_FIT em 4096 palavras, disco com C_LOOK, log em JSONL e o diagrama de Gantt como recurso extra. Esses valores são os padrões do programa: uma chave que falta no hv.conf assume o valor do escopo. Como política de referência implementei o FCFS, que pode ser escolhido com -p FCFS sem mexer no arquivo de configuração.

2. Arquitetura

O programa segue a estrutura de diretórios do enunciado. Cada módulo tem um par .h/.c, os tipos são opacos (a struct só existe no .c) e todo estado fica em structs passadas por ponteiro, sem variável global mutável. A Figura 1 mostra as camadas e quem usa quem.

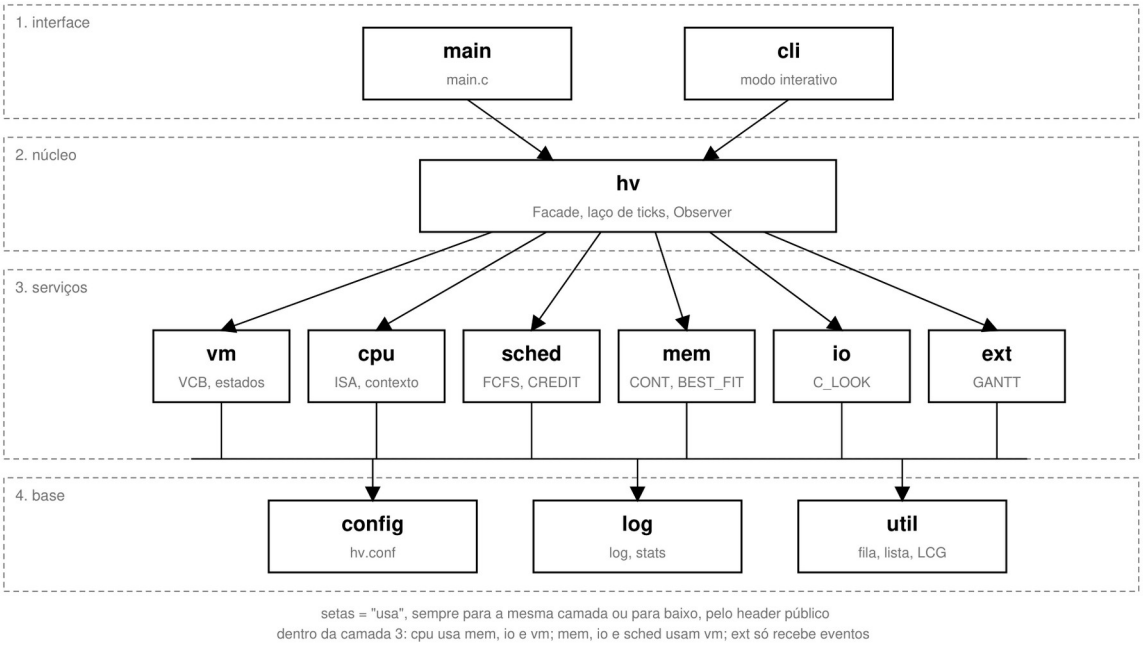


Figura 1. Módulos do MiniHV e dependências entre eles.

A parte que mais me fez pensar foi como os módulos de baixo avisam o hv de alguma coisa sem incluir hv.h, o que criaria um ciclo. A saída foi o EventSink, definido em events.h: uma struct com um ponteiro de função e um contexto. O vm, o cpu, o mem, o io e o sched recebem um ponteiro para ele quando são criados e chamam event_emit. Quem implementa a função é o hv, que repassa o evento para os inscritos: o log, o GANTT e, nos testes, o próprio teste. É o padrão Observer, só que sem nenhum módulo conhecer o hv.

O hv acabou dividido em dois arquivos. Com tudo junto ele passava de 600 linhas, então hv.c ficou com a inicialização e o laço de ticks, e hv_cmd.c com o relatório e os comandos do

modo interativo. A struct Hv fica num header privado do módulo (hv_internal.h), que nenhum outro módulo inclui. A Tabela 1 resume onde ficou cada padrão de projeto pedido.

Padrão	Onde ficou
Facade	hv.h: única API usada por main.c e cli.c
Strategy	SchedPolicy (sched_policy.h) e AllocPolicy (mem_policy.h)
Factory	sched_policy_create("CREDIT") e mem_policy_create("BEST_FIT")
State	tabela TRANSICAO_VALIDA[de][para] consultada por vm_set_state()
Observer	EventSink + hv_subscribe(); log e GANTT são inscritos
Produtor/consumidor	cada dispositivo tem uma fila de IoReq e atende uma por vez
TAD	Queue e List em util, usadas por sched, io e mem

Tabela 1. Padrões de projeto.

3. Decisões de projeto

3.1 Laço de ticks

Cada chamada de hv_step faz os seis passos do RF-HV-02 na ordem. Um detalhe que só percebi ao conferir o CREDIT à mão: as VMs que ficam prontas no mesmo tick precisam entrar na fila ordenadas por id, mesmo vindo de lugares diferentes (admissão no passo 1, interrupção no passo 2). Por isso elas vão primeiro para um vetor pending, que é ordenado e despejado na fila no fim do passo 2. As VMs preemptadas por fim de quantum só voltam para a fila no passo 3 do tick seguinte, depois das chegadas.

A troca de contexto é de verdade: cada pCPU tem seu próprio banco de registradores e PC. No despacho o cpu copia os registradores do VCB para a pCPU, e quando a VM sai copia de volta. Contei como troca de contexto quando entra numa pCPU uma VM diferente da última que esteve nela. Se a mesma VM é preemptada e volta para a mesma pCPU, não conta.

3.2 CREDIT

O estado da política é uma fila circular de VMs prontas e um vetor de contas indexado pelo id da VM. A conta guarda o saldo, o peso (10 menos a prioridade) e quantos ticks do quantum já foram usados. A VM ganha saldo igual ao quantum (5) na primeira vez que entra na fila. Cada tick em RUNNING tira 1 crédito. No pick_next eu percorro a fila e pego a primeira VM com saldo maior que zero (UNDER); se não houver nenhuma, pego a primeira da fila (OVER).

A interface mínima do enunciado não tinha como avisar a política de duas coisas que o CREDIT precisa: que uma VM terminou (para ela sair da distribuição de créditos) e que passou um tick (para a recarga a cada 30). Acrescentei on_exit, remove (usado no pause) e on_clock. A recarga acontece no começo do passo 3 dos ticks 30, 60, 90 e assim por diante: 30 × pcpus créditos divididos pelo peso, com divisão inteira, e o saldo fica limitado a 60. Entram na divisão as VMs admitidas que não terminaram, inclusive as bloqueadas e pausadas. O enunciado não dizia o que fazer com elas, e achei mais justo não punir quem está esperando E/S.

Outras escolhas: o saldo pode ficar negativo sem limite; uma VM UNDER que fica pronta não tira uma OVER da CPU antes do fim do quantum, porque o enunciado só fala em

preempção ao fim do quantum; e quem bloqueia perde o resto do quantum e volta para o fim da fila quando acorda.

3.3 Memória CONT com BEST_FIT

A memória física é um vetor de int32_t alocado uma vez só. As partições ocupadas ficam numa lista ordenada por endereço, e os buracos não são guardados: são calculados como os espaços entre partições sempre que preciso. Com isso a coalescência de buracos vizinhos sai de graça, porque quando uma partição sai da lista os dois espaços ao lado viram um só. O BEST_FIT recebe os buracos em ordem de endereço e escolhe o menor que cabe, usando comparação estrita para que o empate fique com o menor endereço.

Quando o livre total cabe o pedido mas nenhum buraco cabe, a compactação empurra as partições para o início, move os dados com memmove e atualiza o registrador base de cada VM. Ela devolve quantas partições mudaram de lugar e o hv congela a execução por esse número de ticks: as VMs não mudam de estado, só não executam. Os dispositivos continuam andando. A admissão respeita a ordem de chegada; se a primeira VM em NEW não cabe, as que chegaram depois esperam atrás dela.

A tradução é por base e limite. Qualquer endereço fora de [0, mem_words) gera PROTECTION_FAULT. A busca de instrução também passa pelo mem, com endereço virtual igual ao PC: as palavras 0 a n-1 da partição guardam o opcode de cada instrução, e a CPU usa a instrução já decodificada pelo loader.

3.4 E/S

Cada dispositivo tem uma fila e atende uma requisição por vez. Uma requisição feita no tick t começa no passo 2 do tick t+1 e, com latência L, acorda a VM no passo 2 do tick t+1+L. No disco, a cabeça vai para o cilindro escolhido na hora em que o atendimento começa e a distância entra na latência. O C_LOOK pega o pedido mais próximo com cilindro maior ou igual à cabeça; se não há nenhum acima, salta para o menor pendente e o salto conta como deslocamento. Se uma VM é morta enquanto espera, o pedido dela sai da fila, e se já estava em atendimento o dispositivo termina sem acordar ninguém.

3.5 GANTT

O GANTT fica todo no módulo ext e só recebe eventos. O cpu emite um evento interno EXEC a cada instrução executada, com a pCPU e a VM; o ext guarda isso numa matriz tick × pCPU. Esse evento e o TICK_END não vão para o arquivo de log, senão o log dobraria de tamanho. No fim da simulação (e no comando gantt) o diagrama sai em texto, com uma linha por pCPU, régua a cada 10 ticks e quebra a cada 80, e o out/gantt.csv é gravado. Com enabled = false o hv nem inscreve o ext e nada muda no resto do programa.

4. Testes

Há um executável de teste por módulo em tests/, todos com assert, rodados por make test. Os de unidade cobrem o parser de configuração (inclusive as mensagens com arquivo e linha), o loader, as transições de estado, a escolha de partição e a compactação, a ordem do C_LOOK com as latências e o CREDIT isolado. Os de ponta a ponta usam os arquivos de conf/:

- test_sched compara a sequência de DISPATCH do conf/ta02_1cpu.conf (4 VMs numa pCPU) com a sequência que calculei à mão, que está no README. Faz o mesmo com o FCFS.
- test_hv cobre o TA-01, o TA-04 (espera em NEW e uma compactação que move 2 partições), o TA-05, o TA-06 e o TA-07, rodando hv.conf duas vezes e comparando saída e log byte a byte.
- test_ext é o TA-08: reconstrói a ocupação de cada pCPU só com os eventos DISPATCH e STATE_CHANGE e compara com o gantt.csv, tick a tick.
- test_cli alimenta o modo interativo com um arquivo de comandos.

make valgrind roda o binário em todos os cenários de conf/ e todos os testes no valgrind com --leak-check=full, e o resultado é 0 bytes perdidos (na verdade 0 bytes em uso na saída). O código também compila sem aviso com clang 18. O cenário conf/perf.conf, com 8 VMs por 10.000 ticks, levou 0,05 s numa pasta do Linux.

5. CREDIT comparado com FCFS

Rodei os mesmos cenários com as duas políticas (-p FCFS). A Tabela 2 mostra as médias do relatório final de cada execução.

Cenário	Política	Turnaround	Espera	Resposta	Trocas de contexto	Ticks
hv.conf (7 VMs, 4 pCPUs)	CREDIT	143,3	6,7	0,9	72	535
	FCFS	147,6	11,0	1,6	25	526
ta02_1cpu (4 VMs, 1 pCPU)	CREDIT	76,0	54,0	4,5	20	92
	FCFS	53,5	31,5	31,5	4	92
ta08 (6 VMs, 4 pCPUs)	CREDIT	76,5	7,2	0,2	46	143
	FCFS	78,2	8,8	3,3	19	143
perf (8 VMs, 10.000 ticks)	CREDIT	8763,2	3757,2	2,5	5559	10000
	FCFS	10000,0	5000,0	0,0*	4	10000

*Tabela 2. Médias em ticks. * só 4 das 8 VMs chegaram a rodar.*

No ta02_1cpu o FCFS ganha em turnaround (53,5 contra 76,0). Faz sentido: as quatro VMs têm o mesmo tamanho e não fazem E/S, então fatiar a CPU só atrasa todo mundo. Em compensação a resposta média cai de 31,5 para 4,5, porque com CREDIT nenhuma VM espera mais que 9 ticks para rodar pela primeira vez. É a troca clássica de qualquer política com quantum.

Com 4 pCPUs a diferença fica menor, porque quase sempre há CPU livre. Mesmo assim, no hv.conf a espera média caiu de 11,0 para 6,7. Quem mais ganhou foram as VMs de E/S: a db (prioridade 1) terminou em 94 ticks contra 124 no FCFS, e a web em 36 contra 52. No ta08 a VM web, de prioridade 8, chega em t=5 e no FCFS fica 34 ticks esperando atrás das CPU-bound; com CREDIT espera 6. O preço é o número de trocas de contexto, quase o triplo.

O cenário perf é o mais interessante. Com FCFS as quatro primeiras VMs são laços infinitos, ocupam as 4 pCPUs e nunca saem: as outras quatro passam 10.000 ticks em READY e não rodam nenhuma instrução. A io1, que terminaria em uns 100 ticks, nunca começa. Com CREDIT todas rodam, e a fatia de cada uma segue a ordem dos pesos (Tabela 3).

VM	Peso	CPU com CREDIT	CPU com FCFS
l0	10	8655	10000
l1	9	7973	10000
l2	8	6982	10000
l3	7	6315	10000
io1	6	35 (HALT em t=106)	0
l5	5	4330	0
l6	4	3335	0
l7	1	2375	0

Tabela 3. Ticks de CPU por VM no cenário perf.

A ordem bate, mas a proporção não. Pelos pesos, l0 deveria ter umas 10 vezes a CPU de l7 e teve só 3,6 vezes. Fui atrás do motivo e ele está na divisão inteira: a cada 30 ticks entram 120 créditos e, com sete VMs e soma de pesos 44, as parcelas são 27, 24, 21, 19, 13, 10 e 2, que somam 116. Somem 4 créditos por recarga, enquanto o consumo é de 120. Com o tempo o saldo de todo mundo vai ficando negativo, e quando não existe VM UNDER na fila a escolha vira a ordem da fila, ou seja, round robin. O peso só pesa logo depois de cada recarga. O limite de 60 no saldo também corta parte do que as VMs de peso alto ganhariam. Achei esse resultado bom para entender por que o Xen de verdade tem mais detalhes no CREDIT do que a versão simplificada do enunciado.

6. Limitações

Só implementei o que o escopo 09 usa: CREDIT e FCFS no escalonador, CONT com BEST_FIT na memória e C_LOOK e FCFS no disco. Outros valores nessas chaves dão erro de configuração com arquivo e linha. O binário não cria a pasta out/ (quem cria é o make). Rodando no WSL dentro de uma pasta do Windows a gravação do log fica bem mais lenta, uns 2 s no perf.conf, por causa do sistema de arquivos; numa pasta do Linux o mesmo teste leva 0,05 s.